Untrusted Authors, Trusted Answers

A Calculus of Fidelity-Graded Translations

Anonymous Author(s)

Abstract

Verified translation has two well-studied extremes: prove the translator once (certified compilation), or validate each run of one translator (translation validation). Both treat a single translation in isolation. We study translations as a *graph* — many source languages, several reasoning targets, multiple independently built routes — where the honest answer to "is this translation correct?" differs from edge to edge. We present a calculus of *fidelity-graded translations*: pairs of languages close commuting squares that are checkable per program and compose by pasting; declared fidelity grades compose by weakest link, are *re-established* per run by inline checking, and are *exceeded* by agreement between independently derived routes; and an end-to-end theorem isolates a fundamental asymmetry — witness-carrying answers are self-certifying at the source, while universal answers are where grades, branches, and certificates earn their cost. The compositional core is mechanized in Lean 4.

The calculus is implemented in HURDY-GURDY, a platform of 13 languages and 13 pairs organized around two reasoning hubs, built as a *two-directional experiment in LLM-generated correctness*: independent LLM agents wrote every pair, largely unsupervised, with the architecture's cross-checks as the only semantic gate, and the platform's intended *player* is itself an LLM, handed deterministic, graded, checked moves — a direction we evaluate in a first controlled experiment. All code, and most of this paper, is LLM-generated; the human contribution is the architecture. We report measured per-construct conjoined coverage, branch agreement along dual RISC-V and AArch64 routes, a compliance-derived benchmark decided along both RISC-V routes with machine-derived ground truth, source-level witness replay, certified unreachability re-validated by a formally verified checker (at propagation and at megabyte-certificate scale), a measured escape-rate experiment for the gate, and the defects the architecture caught — including one in that checker's own adapter and three in the coverage instrument itself.

1 Introduction

To answer a question about a program, move the program to where the question is decidable. A C program's reachability question becomes a bit-vector model-checking problem; a Python function's assertion becomes a linear-arithmetic query; a chemical reaction network's coverability question becomes an SMT unrolling. Every such move is a translation, and every translation is a place to be wrong. The classical responses are to *prove* the translator once and for all [12, 13], or to *validate* each of its runs [14, 21, 25]. Both are statements about a single edge.

This paper is about the *graph*. In practice the representations worth reasoning in form a web: several source languages, a few solver-facing targets, more than one way to get from here to there — and, crucially, edges of *honestly different* trustworthiness. A translator generated rule-for-rule from a logic-to-logic mapping is foreseeable from its specification. An optimizing C compiler is not foreseeable and not proved, only pinned. A translator written from an ISA manual and one derived from the same ISA's formal model are neither proved — but they were produced *independently*, and that independence is itself a resource. A theory of the graph must let each edge say what it actually guarantees, compose those statements honestly, and show how several weak edges can corroborate one another into a strong route. Its stance is the one distributed systems take toward processes: a translator is a component that fails, and the failure is to be *contained* — by declared loss, decidable checks, and independent routes — rather than prevented by universal proof.

An experiment in LLM-generated correctness, in both directions. This work is, deliberately, as much an experiment about *who can build and use* such a system as about the system itself. HURDY-GURDY — the platform this paper defines, builds, and measures — exists to show two things at once. First, that LLMs can be *utilized* to produce correct code: every pair — translator, carry-back, tests — was implemented by an independent LLM agent from a one-page brief, largely unsupervised, with the architecture's cross-checks as the only semantic gate; the platform is the experiment's artifact, and §6.6 reports what the gate caught. Second, that LLMs can be *supported* in producing correct conclusions: the platform's intended player is an LLM, which chooses questions and routes but can take no unchecked step — routes of pairs hand the untrusted reasoner deterministic moves whose trust is graded, cross-checked, and, for witness-carrying answers, self-certifying at the source. This paper evaluates the first direction throughout, and reports a first, deliberately small controlled experiment on the second (§6.8): LLM players answering ground-truthed questions through the platform, every verdict carrying a machine-checked evidence artifact where the unaided baseline offers its say-so. Building the instrument and playing it are the same thesis at two scales: trust comes from the architecture, not from the author.

Provenance disclaimer. All of HURDY-GURDY — framework, shared interpreters, pairs, solver plumbing, the evaluation harness, and the Lean mechanization — is LLM-generated code (Claude Opus and Claude Fable agents), produced largely unsupervised under the discipline this paper describes; no human reviewed the code for semantic correctness. Most of this paper's text was likewise LLM-generated. The human contribution is the architecture: the calculus, the contracts the agents were held to, and the decisions about what to build and what to claim. Every quantitative claim in Section 6 is regenerated from the artifact by one script, and the mechanization's axiom audit is re-checked at every build — by design, the reader need not trust the writers, human or otherwise, any further than the paper's own thesis requires.

This paper. We give the graph that theory, and measure an implementation of it. The unit is the *pair* (Section 3): two formally defined languages, a pure translator, pure interpreters for both languages (owned by the languages and shared across all pairs touching them), and a pure *target-to-source interpreter* — the component, usually an afterthought, that carries a target-side behavior back to a source-side behavior. The four functions close a commuting square, and the square commuting *at a given program* is decidable: run both sides, compare under the pair's declared projection. A failure localizes to a step and a named observable.

Squares compose. The pasting of commuting squares is folklore, and the hypothesis it needs when each square commutes only *up to a projection* — the earlier hop's carry-back must not read target observables the later hop discards (Definition 3.6 and Theorem 3.7) — is, as mathematics, a congruence condition whose cousins appear wherever simulations compose vertically across differing observations (Section 7). What we claim for it is its role as an explicit design rule — *declare your loss* — with a syntactic keep/loss computation: a source observable survives a route iff no hop discards what it depends on.

On this base we build the fidelity story (Section 4). Each edge declares a grade — `predicted`, `reproducible`, `checked`, `proved`, `trusted` — naming its guarantee and evidence. Grades compose by weakest link, but on an *assurance-class* chain (universal / per-run / replay / none) that separates the logic of a guarantee from the kind of evidence behind it. Two further mechanisms move assurance where weakest-link says it cannot go: *re-establishment* (Theorem 4.3) — inline square-checking lifts any single run over an opaque hop to per-run faithfulness, which is how a pinned-but-unproved compiler can head a verification route — and *branch corroboration* (Lemma 4.5) — independently derived routes to the same target that agree on the same input corroborate each other beyond either's declared grade, with disagreement localizing to one hop of one route (Lemma 4.4). A trusted-base ledger (§4.4) itemizes exactly what each layer of checking removes from the set of components one must still believe.

The capstone (§4.6) is an asymmetry we believe deserves to be a design principle. Answers that carry witnesses — REACHABLE, SAT — are *self-certifying at the source*: carry the witness back through the route's target-to-source interpreters and replay it in the source interpreter; soundness then rests on interpreter adequacy alone, with every translator and solver demoted to a discovery heuristic (Theorem 4.8). Universal answers — UNREACHABLE, UNSAT — admit no witness, and are precisely where fidelity grades, branch agreement, and independently checked certificates are the only currencies (Theorem 4.9). A translation architecture should therefore make witness carry-back a first-class component of every edge — cashing in the free half of trust — and spend its proof effort exclusively on the other half.

Coverage, and honesty about partiality. Fidelity is gameable by triviality (a one-instruction pair is vacuously `proved`); coverage is gameable by unsoundness. We require their per-construct *conjunction* against spec-enumerated inventories (Definition 4.6), with all rejection typed and enumerable, and we prove the widening discipline sound: additive extensions preserve all prior verdicts (Proposition 4.7), so coverage ratchets monotonically over a project's history.

The instantiation. The calculus is implemented as HURDY-GURDY, a platform of 13 registered languages and 13 pairs (Section 5): C, RISC-V, AArch64, Wasm, eBPF, EVM, Python, Sail, chemical reaction networks, SMILES molecular graphs, and two reasoning hubs — BTOR2 (bit-level) and SMT-LIB (theory-level) — plus dual, independently derived routes RISC-V⟶BTOR2 and AArch64⟶BTOR2 (manual-derived vs. Sail-model-derived), each pair agent-built as framed above with the square oracle, external differentials (a Sail-generated ISA simulator, a pinned CPython), solver corroboration, and branch cross-checks as the only police. Section 6 reports the resulting capability snapshot — per-pair conjoined coverage, composed route coverage, branch-agreement runs down to SMT-LIB, end-to-end case studies with source-level witness replay, certified unreachability, performance, and the defects the architecture caught along the way, each localized as Corollary 3.8 promises.

Contributions.

- A calculus of pairs: commuting squares with declared projections, decidable per-program checking with step/observable localization, and a pasting theorem whose projection-compatibility hypothesis makes cumulative loss compositional (Section 3).
- A fidelity algebra: grades vs. assurance classes, weakest-link composition, per-run re-establishment, branch corroboration under an explicit diversity assumption, and a trusted-base ledger (Section 4) — with the compositional core of both sections mechanized in Lean 4 (§4.7).

- The existential/universal asymmetry as an architectural principle, with the end-to-end soundness theorems that express it (§4.6).
- A measured, LLM-built instantiation: 13 agent-built pairs over 13 languages and two solver hubs, evaluated for conjoined coverage, branch agreement, end-to-end witness replay, certified unreachability re-validated by a formally verified checker, and defects caught (Sections 5 and 6).

This document is a dated capability snapshot (July 2026) of a system whose coverage ratchets monotonically by construction; every number in Section 6 is regenerable from the accompanying anonymized artifact at this snapshot.

2 Overview: one question, two routes

Consider a small C program: a loop over an array with an index computation, and a question — can `x == 42` hold at the failure label? We narrate the route this question takes through the platform; Section 3 then makes the definitions precise.

The spine. The program first moves from C to RISC-V. The translator on this edge is a *pinned* C compiler: a digest-locked gcc with recorded flags. We can replay it bit-for-bit, but nobody can foresee or prove its output — its honest grade is `reproducible`, determinism without meaning. The RISC-V program then moves to BTOR2, a bit-vector transition-system logic that model checkers consume, and from there across a rule-for-rule bridge to SMT-LIB, where bit-vector solvers decide the reachability query.

Each of these moves is a *pair*: translator, source interpreter, target interpreter, and a target-to-source interpreter that carries behaviors backwards. When the RISC-V-to-BTOR2 translator runs, the platform can immediately check it *on this very program*: interpret the RISC-V program directly (the left edge of a square), interpret the BTOR2 translation and carry the resulting trace back to RISC-V observables (the other three edges), and compare, step by step, on the declared observables — program counter, registers, halt flag. That check passing is what the edge's `checked` grade *means*. Note carefully what it covers: every hop *from the compiled binary onward*. The C hop itself has no square — there is no C interpreter and no ISA-to-C carry-back — so the verified portion of the route begins at the binary; the pinned compiler contributes reproducibility, re-checked separately by a corpus-level differential against an independent C verifier (Section 5).

The branch. RISC-V reaches BTOR2 twice. One translator was written against the prose ISA specification; a second route factors through Sail, deriving its semantics from the official RISC-V Sail model. On the segment where they diverge — from RISC-V to BTOR2 — the two routes share no translator and no carry-back; below the reconvergence they share the hub bridge, so it is the diverse prefix that agreement corroborates (§4.3). The platform runs the question down both, and the answers must agree. If they do, two independently derived encodings of RISC-V semantics corroborate each other — evidence strictly stronger than either route's own grade (Lemma 4.5). If they disagree, at least one route has a defect, and the per-hop squares localize it: which hop, which step, which observable (Lemma 4.4 and Corollary 3.8). The same dual-route structure exists for AArch64 (Arm manual vs. Arm Sail model).

The answer comes home. Suppose the SMT solver answers SAT: the label is reachable, with a model. The model is not the answer the user asked for — it is a valuation of SMT variables. The route's carry-backs now run in reverse: the SMT model becomes a BTOR2 witness, the witness a RISC-V trace, the trace an initial memory and register state — concretely, the inputs of the original C program. The platform then *replays* those inputs in the source interpreter and observes `x == 42` actually happen. At that moment the user's trust obligations collapse: even if every translator, both routes, and the solver were wrong, the replayed run exhibits the behavior (Theorem 4.8). Witness-carrying answers are self-certifying; only the interpreter that replays them — itself differentially tested against an independently generated ISA simulator — remains to be believed.

Had the solver answered UNSAT, no replay could vouch for it. That is where the grades, the branch agreement, the multi-engine corroboration, and the independently checked certificates carry the weight (Theorem 4.9) — and where the platform spends its assurance budget.

Beyond one spine. Nothing above is specific to C or RISC-V. The same square contract admits eBPF, Wasm, and EVM programs into the BTOR2 hub; Python functions and chemical reaction networks reach the SMT-LIB hub directly; a SMILES molecular graph reaches a molecular-formula target with no solver at all (a *compile pair* — the calculus does not care that the languages are far from computation, only that both have defined meaning). Each pair declares what it preserves (its projection), what it rejects (typed, enumerable `unsupported`), and its grade; the registry composes routes and reports each route's composed coverage, grade, and loss. The rest of the paper makes each of these words precise, then measures the system.

3 A calculus of pairs

This section makes the informal contract of Section 2 precise: languages, behaviors, and projections (§3.1); pairs and their commuting squares (§3.2); and the pasting theorem that lets squares compose into routes, together with the projection-compatibility condition that pasting actually requires (§3.3). Fidelity, coverage, and the end-to-end guarantee build on this base in Section 4.

3.1 Languages, behaviors, projections

Definition 3.1 (Language)**.** A *language* A is a triple $(\mathrm{Prog}_A, F_A, [\![\cdot]\!]_A)$ where Prog_A is a decidable set of *programs*; F_A is a finite set of named *observable fields*, each $f \in F_A$ with a value domain V_f; and $[\![\cdot]\!]_A : \mathrm{Prog}_A \rightharpoonup \mathrm{Beh}_A$ is the *reference semantics*, a partial function into behaviors. An *observable state* is a total map $\sigma \in \mathrm{Obs}_A$, where $\mathrm{Obs}_A = \prod_{f \in F_A} V_f$; a *behavior* is a finite sequence of observable states, $\mathrm{Beh}_A = \mathrm{Obs}_A^*$, recorded *post-step*: the i-th state is the observable state after the i-th transition.

The only admission criterion for a language is that $[\![\cdot]\!]_A$ is *definable* — an ISA manual, a logic's model theory, a reaction-network semantics, and a molecular-graph notation all qualify. Nothing in this section assumes languages are executable or even computational.

The definition also fixes the calculus's scope, and we state the restrictions plainly: behaviors are *finite* and the reference semantics is a *function*, so nondeterminism, divergence, reactive I/O, and refinement-style behavior *inclusion* are outside this calculus — faithfulness is projected trace *equality*. That is the right notion for the near-structure-preserving translations the platform builds and checks, and the wrong one for optimizing compilation, whose home is simulation; accordingly, the one genuinely optimizing hop in the instantiation carries no square at all and is handled differently (§4.2, Section 5). Languages whose official semantics is not a function (C, with its unspecified orders and undefined behavior) enter through a determinized fragment, with the gap policed differentially (Section 5).

A program here is a *closed* configuration: free inputs, when present, are part of p (a program together with an input valuation). Open programs — programs quantified over their inputs — reappear in §4.6, where a solver decides over all inputs and a witness closes one.

Definition 3.2 (Interpreter)**.** An *interpreter* for A is a computable partial function $I_A : \mathrm{Prog}_A \rightharpoonup \mathrm{Beh}_A$ that is *pure*: identical input yields byte-identical output, on every host. Outside its domain it fails with a typed `unsupported` verdict naming the construct; $\mathrm{dom}(I_A)$ is A's *covered fragment*.

Assumption 1 (Interpreter adequacy)**.** $I_A(p) = [\![p]\!]_A$ for all $p \in \mathrm{dom}(I_A)$.

Adequacy is an assumption, not a theorem, and we keep it visible: it is the irreducible residue in every trusted-computing-base computation of §4.4. It is discharged *empirically*, by differential testing against an independently produced oracle for the same semantics (the Sail-generated RISC-V simulator; the pinned CPython runtime; Section 5), and its failure modes are cross-checked by the same machinery that checks translators.

Definition 3.3 (Projection)**.** A *projection* over A is a subset $\pi \subseteq F_A$. It lifts to states by restriction, $\pi(\sigma) = \sigma\restriction_\pi$, and to behaviors pointwise, $\pi(\sigma_1 \cdots \sigma_n) = \pi(\sigma_1) \cdots \pi(\sigma_n)$ (in particular, projected behaviors of different lengths are unequal). Write $b \equiv_\pi b'$ for $\pi(b) = \pi(b')$.

3.2 Pairs and the commuting square

Definition 3.4 (Pair)**.** A *pair* $P : A \to B$ is a tuple $(A, B, T, \Lambda, \pi_P)$ where $T : \mathrm{Prog}_A \rightharpoonup \mathrm{Prog}_B$ is the *translator*; $\Lambda : \mathrm{Beh}_B \rightharpoonup \mathrm{Beh}_A$ is the *target-to-source interpreter*, which re-expresses a target behavior as a source behavior; and $\pi_P \subseteq F_A$ is the pair's *declared projection* — exactly the source observables it promises to preserve. T and Λ are pure (as in Definition 3.2); I_A and I_B are owned by the languages and shared by every pair that touches them. The pair's domain $\mathrm{dom}(P)$ is the set of p with $p \in \mathrm{dom}(I_A) \cap \mathrm{dom}(T)$, $T(p) \in \mathrm{dom}(I_B)$, and $I_B(T(p)) \in \mathrm{dom}(\Lambda)$.

Definition 3.5 (Faithfulness)**.** P is *faithful at* $p \in \mathrm{dom}(P)$ when the square commutes at p, i.e. when $\pi_P(I_A(p))$ equals $\pi_P(\Lambda(I_B(T(p))))$:

$$\begin{tikzcd} p \arrow[r, "T"] \arrow[d, "I_A"'] & T(p) \arrow[d, "I_B"] \\ I_A(p) & \Lambda(I_B(T(p))) \arrow[l, "\equiv_{\pi_P}"'] \end{tikzcd}$$

P is faithful on $C \subseteq \mathrm{dom}(P)$ when it is faithful at every $p \in C$.

Three remarks. First, everything in Definition 3.5 is computable: faithfulness at a given p is a *decidable, executable check* — run both sides, compare under π_P. We call this check the *square oracle*; in fault-tolerance terms it is an acceptance test in the sense of recovery blocks [26], turning a silently wrong — in effect Byzantine — translator into a *fail-stop* component [32]. Second, when the check fails it fails at a *first* position: a least step index i and a field $f \in \pi_P$ on which the two sides differ. The oracle reports (i, f) — a translator (or interpreter, or carry-back) defect, localized to a step and a named observable. Third, the projection is part of the contract, not a tuning knob: π_P states what "preserves meaning" promises for this pair, and $F_A \setminus \pi_P$ — the pair's *loss* — states what it does not.

The carry-back Λ is what makes a target-side result *mean* something at the source: a solver counterexample or a target trace is re-expressed as a source behavior. It is pair-specific because the correspondence it encodes is pair-specific; it is also why the square's oracle is as expressive as the translator it checks.

3.3 Pasting: when squares compose

Two pairs compose when the middle language is shared: $P_1 = (A, B, T_1, \Lambda_1, \pi_1)$ and $P_2 = (B, C, T_2, \Lambda_2, \pi_2)$ yield the candidate composite

$$P_2 \circ P_1 \;=\; (A,\ C,\ T_2 \circ T_1,\ \Lambda_1 \circ \Lambda_2,\ \pi)$$

for a projection $\pi \subseteq \pi_1$ to be determined, with $\mathrm{dom}(P_2 \circ P_1)$ the evident composite domain. Folklore says the squares paste. They do not paste for free: P_2 guarantees agreement of the two B-behaviors only *up to* π_2, and Λ_1 consumes whole B-behaviors. If Λ_1 reads a B-field that P_2 discards, the outer rectangle can fail while both inner squares hold. The missing condition is that Λ_1 must not look outside what P_2 preserves:

Definition 3.6 (Support)**.** Λ_1 is $(\pi_2 \Rightarrow \pi)$*-supported* when for all $b, b' \in \mathrm{dom}(\Lambda_1)$: $\pi_2(b) = \pi_2(b')$ implies $\pi(\Lambda_1(b)) = \pi(\Lambda_1(b'))$. Equivalently: $\pi \circ \Lambda_1$ descends to a well-defined map on π_2-equivalence classes — a congruence condition, matching the mechanization exactly.

Theorem 3.7 (Pasting)**.** *Let $p \in \mathrm{dom}(P_2 \circ P_1)$, let P_1 be faithful at p, let P_2 be faithful at $T_1(p)$, let $\pi \subseteq \pi_1$, and let Λ_1 be $(\pi_2 \Rightarrow \pi)$-supported. Then, writing $r = T_2(T_1(p))$, the composite $P_2 \circ P_1$ is faithful at p with respect to π:*

$$\pi(I_A(p)) \;=\; \pi\big(\Lambda_1(\Lambda_2(I_C(r)))\big).$$

Proof. Write $q = T_1(p)$. Then $\pi(I_A(p)) = \pi(\Lambda_1(I_B(q)))$ since $\pi \subseteq \pi_1$ and P_1 is faithful at p. Faithfulness of P_2 at q gives $\pi_2(I_B(q)) = \pi_2(\Lambda_2(I_C(r)))$, so the support condition applies to the two B-behaviors and yields $\pi(\Lambda_1(I_B(q))) = \pi(\Lambda_1(\Lambda_2(I_C(r))))$. Chain the equalities. (Details: Appendix A.) □

Composed keep and loss. In the implementable special case — Λ_1 *fieldwise* and step-aligned, each source field f computed from a dependency set $\mathit{deps}(f) \subseteq F_B$ of target fields — the support condition has a syntactic reading, and the largest π satisfying Theorem 3.7 is

$$\mathrm{keep}(P_2 \circ P_1) \;=\; \{\, f \in \pi_1 \mid \mathit{deps}(f) \subseteq \pi_2 \,\},$$

a source field survives the route iff P_1 keeps it and it is computed only from target fields P_2 keeps. Dually $\mathrm{lost}(P_2 \circ P_1) = F_A \setminus \mathrm{keep}(P_2 \circ P_1)$: route loss is the union of per-hop losses, each pulled back along the carry-backs. This is the formal content of the platform rule that a route must *declare its cumulative loss*: the observables a destination can still speak about are exactly those no hop discarded. (When Λ_1 changes step granularity — one source step spanning several target steps, as in C-to-ISA — the same statement holds with *deps* taken over the spanned window; Appendix A.)

Corollary 3.8 (Localization)**.** *Under the support condition, if the composite square fails at p (both sides defined), then P_1's square fails at p or P_2's square fails at $T_1(p)$. Inductively, along a route $R = P_n \circ \cdots \circ P_1$ satisfying the pairwise support conditions, a failing outer rectangle implies a failing inner square at some hop i — and running the square oracle hop by hop finds the least such i, then the least failing step and field.*

Corollary 3.8 is the debugging story of the whole platform: a divergence anywhere along a route is never a mystery about the route; it is an indictment of one hop, one step, one observable. It is also what disagreement between *routes* will lean on in §4.3.

Routes. The registry of languages and pairs forms a directed multigraph; a *route* $R : A \to Z$ is a path through it, with composed translator T_R, composed carry-back Λ_R, composed projection $\mathrm{keep}(R)$, and faithfulness given by iterating Theorem 3.7. Purity composes too, and gives caching:

Proposition 3.9 (Determinism and caching)**.** *If every component function of every hop of R is pure, then T_R and Λ_R are pure, and a content-addressed cache keyed by the input hash and the component versions returns, on a hit, exactly what recomputation would return — across the whole route. If some hop is impure, re-running the route on the same input and diffing bytes at every hop (*twice-and-diff*) can detect it and localize it to the first hop whose bytes differ — a sound but incomplete detector: two coinciding runs of an impure hop pass.*

4 Fidelity, coverage, and the end-to-end guarantee

Section 3 says what it means for one translation, or one route, to be faithful. This section grades *how well we know* it is faithful (§4.1), shows how the grades compose — weakest link, per-run re-establishment, and branch corroboration (§4.2–§4.3) — accounts for what each layer of cross-checking removes from the trusted base (§4.4), pairs fidelity with its anti-triviality companion, coverage (§4.5), and assembles everything into the end-to-end guarantee a user actually receives (§4.6).

4.1 Fidelity grades and assurance classes

A pair declares a *fidelity grade* from the set G = {`trusted`, `reproducible`, `checked`, `predicted`, `proved` }, each grade naming both a guarantee and the *kind of evidence* that establishes it: `predicted` — the translator's output is derivable byte-for-byte from a written specification (evidence: an audit that re-derives the bytes); `proved` — a machine-checked certificate that the square commutes, re-checked by an independent checker (evidence: the certificate and the checker's pedigree); `checked` — the square oracle runs and passes on every translation actually performed (evidence: this run's oracle verdict); `reproducible` — determinism only, via a digest-pinned toolchain (evidence: replayability, no meaning guarantee); `trusted` — nothing.

The grades order by evidence strength, but `predicted` and `proved` are incomparable in *kind* — one is foreseeable, the other re-checkable — and nothing below is gained by forcing them into a line. What composition actually needs is the logical *form* of the guarantee, and that is totally ordered:

Definition 4.1 (Assurance class)**.** The *assurance classes* are the chain $\mathbb{A}$: `none` < `replay` < `perrun` < `universal`, and *class* : $G \to \mathbb{A}$ maps `trusted` ↦ `none`, `reproducible` ↦ `replay`, `checked` ↦ `perrun`, and `predicted`, `proved` ↦

universal. The classes denote guarantee predicates for a pair P with covered fragment C_P: universal — P is faithful at every $p \in C_P$; perrun — P is faithful at every p on which the square oracle has run and passed (in particular, at the program of the present run); replay — T is pure, nothing about meaning; none — nothing.

Separating G from $\mathbb{A}$ resolves a wrinkle the informal five-tier story leaves open (what is the "meet" of `predicted` and `proved`?): grades differ in evidence, classes in logic, and only classes compose. A route with one `predicted` hop and one `proved` hop is universal end to end, as it should be.

Two honesty notes on the mapping. First, grade *declarations* are themselves trusted inputs: only `checked` is mechanically enforced by the framework (the oracle runs); *class*(`predicted`) = universal additionally presumes that the audited specification maps rules to *semantics* — a translator faithful to a wrong rule table is `predicted` and universally unfaithful, and our own evaluation exhibits a `predicted` edge that was wrong on covered inputs until a coverage probe caught it (§6.6). Declarations therefore sit in the trusted-base ledger like any other component. Second, `proved` as instantiated in §6.5 certifies a *per-question* verdict — universal over that question's inputs, per-run over questions — not the pair's square for all programs.

4.2 Composition: weakest link, and per-run re-establishment

Proposition 4.2 (Weakest link)**.** *Let route $R = P_n \circ \cdots \circ P_1$ satisfy the support conditions of Theorem 3.7. If every hop's guarantee holds, then the composite guarantee of class* $\min_i$ *class*(P_i) *holds for R (with C_R the composite fragment), and no higher class is entailed by the hop guarantees alone.*

The proof is immediate from Definition 4.1 and Theorem 3.7: universal statements conjoin into universal statements over the composite fragment; a per-run statement at hop i caps the conjunction at the programs the oracle actually saw; a replay hop caps it at purity. Optimality is by the evident counterexamples. Statically, then, *a route is as faithful as its weakest hop* — but the static story is not the whole story, because every pair carries its own oracle:

Theorem 4.3 (Per-run re-establishment)**.** *Fix a run of route R on program p, with intermediate programs $q_0 = p$, $q_i = T_i(q_{i-1})$. If on this run the square oracle runs and passes at every hop i (on q_{i-1}), and the support conditions hold, then R is faithful at p — regardless of the hops' static grades. In particular a* `reproducible` *or even* `trusted` *hop that is dynamically validated at q_{i-1} contributes, for this run, exactly what a* `checked` *hop contributes.*

Proof. Each passed oracle check *is* faithfulness at q_{i-1} (Definition 3.5 is decidable and the oracle decides it); apply Theorem 3.7 inductively. □

Theorem 4.3 applies exactly to hops whose squares run. That bounds what it can say about an opaque, pinned C compiler at the head of a route: the C hop has no square (no C interpreter, no ISA-to-C carry-back), so what the inline oracles lift, per run, is the route *from the compiled binary onward*. The compiler itself contributes `reproducible` — honest: nobody can foresee or prove its output — plus a separate, corpus-level differential against an independent C verifier (Section 5); questions on the spine are accordingly stated over ISA observables. The declared route grade remains the weakest-link meet — re-establishment is per-run evidence, never a static promotion, and never coverage for a hop with no oracle.

4.3 Branch corroboration

The registry graph is not a line: one source may reach one destination by several routes. Let $R_1, R_2 : A \to Z$ with composed keep-sets K_1, K_2 and $K = K_1 \cap K_2$, sharing the endpoint interpreters (language-owned) but no translator or carry-back.

Lemma 4.4 (Disagreement localizes)**.** *If R_1 and R_2 disagree at p under K, at least one route is unfaithful at p, and Corollary 3.8 pins the failure to a hop, a step, and a field of that route.*

Lemma 4.5 (Agreement corroborates)**.** *If R_1 and R_2 agree at p under K, then either both are faithful at p (under K), or both are unfaithful with* identical projected error*: defects in independently built components producing the same wrong K-behavior at p.*

Both are immediate: two faithful routes each equal $K(I_A(p))$, hence each other. What Lemma 4.5 buys depends on an assumption we state rather than hide:

Assumption 2 (Diversity)**.** Corroborating routes share no translator or carry-back *on the segment where they diverge* (up to the language at which they reconverge), and the diverse translators are derived from *independent semantic artifacts* — e.g. RISC-V reaches BTOR2 once via a translator written from the prose ISA specification and once via the Sail formal model; likewise AArch64 from the Arm manual vs. the Arm Sail model.

The scoping matters, and independence must be argued, not assumed. Below the reconvergence, routes may share hops — in the platform both RISC-V→SMT-LIB routes share the BTOR2→SMT-LIB bridge — and shared emission libraries are shared failure surface, so agreement corroborates the diverse prefix and nothing beneath it. The evaluation therefore also runs a *disjoint-decision* variant of every branch question (§6.2): the direct route's BTOR2 system decided natively by btormc, against the Sail-derived route through the bridge and Z3, so the diverse segment extends through the decision procedure and the shared bridge drops out of the corroborating pair entirely; what remains shared is the emission library (the site of the cross-route incident below) and

Table 1. The trusted-base ledger: what each layer of cross-checking removes from the set of components a verdict still assumes correct.

Layer that ran	**Removed from** TCB	**Still in** TCB
bare route + solver	—	all T_i, Λ_i, all interpreters, solver
+ square oracle at every hop (Theorem 4.3)	every T_i	the Λ_i, the interpreters, solver
+ branch agreement (Lemma 4.5, Assumption 2)	the diverse prefix's Λ_i (cross-corroborated)	shared endpoint interpreters, shared suffix hops and emission libraries, solver; common-mode residue
+ independent solver corroboration	the single solver	the agreeing engines' common failure modes
+ certificate check (`proved`; e.g. DRAT/LRAT)	the solver entirely	the certificate checker (formally verified, in the `cake_lpr` instantiation), plus the bit-blaster's BV→CNF step
+ source-level witness replay (§4.6)	*everything above*	I_A adequacy (Assumption 1) + the replay harness

the language-owned endpoints. Correlated failures between nominally independent implementations are, moreover, the empirical norm [10]; our own history exhibits both a cross-route common-mode failure through a shared emitter and a shared-misreading blind spot (§6.6), and translators written by LLM agents plausibly share model-family failure modes.

Under Assumption 2, the residual failure mode of an agreeing branch is a *common-mode failure*: independently produced encodings of the same semantics, wrong in the same observable way on the same program. We do not assign this a probability; we account for it structurally, in the trusted base ledger below. This is how the platform manufactures assurance out of hops that are individually only `checked` — dual modular redundancy whose comparator, unlike a voter, localizes: agreement of independently derived routes is evidence that neither route's grade alone provides.

4.4 The trusted-base ledger

Define the *trusted computing base* $\mathsf{TCB}(v)$ of a verdict v as the set of components whose correctness v's soundness still assumes after all checks that ran. Each cross-check layer strictly shrinks it (Table 1); the last two rows are not interchangeable, and the difference is the central asymmetry of the whole design.

4.5 Coverage, and the ratchet

Fidelity alone is gameable by triviality: a pair handling one instruction is vacuously `proved`. Its companion axis is measured against a yardstick the implementer does not choose:

Definition 4.6 (Coverage). Fix for each language a *construct inventory* K_A enumerated from its specification (opcodes, operators, syntactic forms). A construct $k \in K_A$ *counts* for pair P iff P covers it (k's probe programs lie in $\mathsf{dom}(P)$) *and* P is faithful on those probes: $\mathsf{cov}(P) \subseteq K_A$ is the per-construct *conjunction* of covered and faithful. Route coverage $\mathsf{cov}(R)$ counts k iff its lowering survives — covered and faithful — at every hop.

Coverage is gamed by unsoundness (accept everything, translate it wrongly) exactly as fidelity is gamed by triviality (translate one thing, prove it); the conjunction closes both routes to a vacuous "pass." Everything a pair rejects must be a *typed* `unsupported`, so the uncovered set is itself enumerable and auditable — partiality is fail-fast, never silent.

Coverage then grows by a discipline with a soundness theorem:

Proposition 4.7 (Ratchet). *Call an update $I' \sqsupseteq I$ (likewise T, Λ) an* extension *when* $\mathsf{dom}(I') \supseteq \mathsf{dom}(I)$ *and* $I' \restriction_{\mathsf{dom}(I)} = I$. *Under extensions: every faithfulness verdict at every* $p \in \mathsf{dom}(I)$ *is preserved;* cov *is monotone; and prior evidence (oracle passes, differentials, agreement records) never needs re-earning. An update that is* not *an extension can silently invalidate any of these, and must instead bump the component's version and re-validate every dependent pair.*

The implementable extension check is byte-identity of all old outputs across the update — the same twice-and-diff mechanism as Proposition 3.9, run across versions. This is precisely the repo's widening protocol (additive interpreter bumps, versioned events, dependent-pair re-validation), which Section 6 measures over the project's own history.

4.6 The end-to-end guarantee

Finally, what does a user hold at the end? Let $R : A \to Z$ be a route into a *reasoning language* Z (one a solver consumes: a transition-system logic, an SMT theory). The user asks a question about an *open* source program $p(x)$ — free inputs x — and a condition φ over $\mathsf{keep}(R)$-observables: is φ reachable? The solver decides at Z; its answer comes back in one of two logical shapes, and they differ fundamentally in what must be trusted.

Theorem 4.8 (Existential answers are self-certifying). *Suppose the solver returns* REACHABLE *with witness* w, *the composed carry-back* Λ_R *re-expresses* w *as an input valuation* x_0 *(and claimed trace), and replaying* $I_A(p(x_0))$ *exhibits* φ. *Then* φ *is truly reachable in* $[\![p]\!]_A$ *— with* $\mathsf{TCB} = \{$*Assumption 1 for* $I_A\}$ *plus the small fixed replay harness the judgment runs through (the projection, the* φ*-evaluator, and the substitution* *closing p), and nothing else. Every translator, carry-back, hub interpreter, and solver on R is a* discovery *device only: a defect in any of them can cause a missed or non-replaying witness, never a false* REACHABLE.

Proof. The replay is a run of I_A itself on a concrete closed program $p(x_0)$; by Assumption 1 its behavior is $[\![p(x_0)]\!]_A$, and φ was observed on it. No other component's output enters the final judgment. □

Theorem 4.9 (Universal answers need the machinery)**.** *Suppose the solver returns* UNREACHABLE *(within declared unrolling/step bounds k). If (i) every hop of R is faithful at every instance $p(x)$ — the* universal*-class guarantee on the fragment containing p's constructs, and exactly the hypothesis the mechanization takes; (ii) the support conditions of Theorem 3.7 hold and φ mentions only* keep(R)*; (iii) the verdict is certified by an independently checked certificate or corroborated across independent engines; and (iv) the symbolic artifact the solver consumes denotes the family of closed translations the squares check — translation commutes with input specialization, an obligation each reasoning pair discharges by construction of its encoding, and under which the mechanization derives every per-instance translation from the one open translation the platform actually performs (*`universal_from_open_artifact`*, §4.7) — then no input drives $[\![p]\!]_A$ to φ within k steps, with* TCB *as in the corresponding row of the ledger (§4.4).*

One caution on hypothesis (i): per-run oracle evidence combined with branch agreement (Lemma 4.5, Assumption 2) *raises confidence* in it but does not entail it — oracle passes cover the finitely many closures actually run, and agreement bounds the residual failure to common-mode rather than excluding it. Universal verdicts are precisely where the universal tiers earn their keep; evidence below that tier is corroboration, not entailment.

The asymmetry between Theorems 4.8 and 4.9 is, we argue, the right lens on the entire design: *for questions whose answers carry witnesses, trust is nearly free* — carry the witness back and replay it at the source; the platform's graded tiers, branches, and certificates exist for the other half, the universal answers where no witness can vouch for the verdict. This is the end-to-end argument [30] transplanted to translation: the check that settles a witness-carrying answer belongs at the endpoint — the source interpreter — and what the intermediate hops provide is, for that half, an optimization; only for universal answers, where no endpoint check can exist, does hop fidelity become the guarantee itself. A system that makes the carry-back Λ a first-class, per-pair component is a system in which the cheap half of trust is always cashed in; a system that grades and stacks cross-checks is one in which the expensive half is bought consciously, in labeled increments, with the bill itemized in the ledger of §4.4.

4.7 Mechanization

The compositional core of Sections 3 and 4 is mechanized in Lean 4 (core library only, no mathlib; ~820 lines, in the artifact): the pasting theorem with the support condition, localization, the assurance-class chain and weakest link (sound half), per-run re-establishment, both branch lemmas, the ratchet (verdict preservation and coverage monotonicity under extensions), both end-to-end theorems, the *specialization obligation* of Theorem 4.9(iv) — with the theorem that it discharges the per-instance-translation hypothesis from the single open translation the platform performs — and the *route telescope* — routes of arbitrary length as a dependently-typed chain of pairs over interpreter-bundled languages, with the paper's "pairwise support conditions" as a recursive coherence predicate, and the n-ary pasting and localization theorems proved by induction through the binary ones (the canonical head-projection choice at each junction is shown lossless by a reprojection lemma). Three modeling choices mirror the paper exactly. Components are Lean functions, so *purity is by construction* and Proposition 3.9 is definitional; partiality is `Option`, with `unsupported` as `none`; and projections are generalized from field subsets to arbitrary observable-compression maps, with "$\pi \subseteq \pi_1$" becoming *factoring*. Because every theorem takes each component-output witness as an explicit hypothesis, a statement's trusted base is literally its hypothesis list — Theorem 4.8 in Lean assumes source-interpreter adequacy and nothing else, and its proof term is three tokens. The axiom audit is re-checked at every build: the branch lemmas, the ratchet, and Theorem 4.8 are axiom-free; pasting, weakest link, Theorem 4.9, and the specialization theorem use only `propext`; the one classical proof is Corollary 3.8 (an unfaithful route names no witness by itself). Deliberately not mechanized: the fieldwise loss computation and retiming case of Appendix A (syntactic side conditions over a concrete carry-back representation), the optimality halves of Proposition 4.2 (meta-statements over models), and the grade set G itself — evidence provenance is not a mathematical object, which is the point of separating G from $\mathbb{A}$. The telescope theorems' audit adds only `Quot.sound` (from structural-recursion equations); localization remains the sole classical proof.

5 Instantiation: the platform

The calculus is implemented as HURDY-GURDY, a Python platform whose structure mirrors Section 3 exactly: a *framework* that holds no pair semantics, per-language *shared interpreters*, and per-pair translators and carry-backs. This section describes what exists at the snapshot; Section 6 measures it.

5.1 Framework

The framework provides the registry (the language/pair graph with deliverable status); the content-addressed cache of Proposition 3.9; the generic square oracle — align $I_s(p)$

against $\Lambda(I_t(T(p)))$ under π, with step/observable localization —; the route enumerator and route runner (routes are enumerated and reported with composed grade, coverage, and loss — never chosen; choosing is the caller's job); the coverage harness (construct-inventory extraction, typed-`unsupported` histograms); the route grader (composed determinism, composed coverage, and branch-agreement measurements, run on merge); and the solver/checker plumbing. Everything except the solver call is byte-deterministic, and a twice-and-diff harness enforces Proposition 3.9 repo-wide.

5.2 Languages and reasoning hubs

Thirteen languages are registered, each with a specification-anchored formal semantics and (for eleven of them, at the snapshot) a built shared interpreter: the ISAs RISC-V (RV64IMC), AArch64 (a growing A64 slice), eBPF, Wasm, and EVM; C (whose "interpreter" role is played differentially, see below); Python (a pinned CPython restricted by an AST allow-list to an integer subset — the loader rejects, CPython runs); Sail (an executable slice of the Sail-derived RISC-V and Arm semantics); chemical reaction networks (a discrete Petri-net stepper); SMILES molecular graphs and molecular formulas; and the two *reasoning hubs*: BTOR2, a bit-vector/array transition-system logic consumed by hardware model checkers, and SMT-LIB (QF_ABV and QF_LIA fragments).

Reasoning languages own, beyond an interpreter, a solver inventory (BtorMC and Pono for BTOR2; Z3, Bitwuzla, Boolector, cvc5, Yices2 for SMT-LIB) and a witness-checker inventory: interpreter replay of BTOR2 `.wit` witnesses, model evaluation by the deterministic SMT-LIB evaluator, multi-engine corroboration, and a bit-blasted certificate pipeline for `proved`-tier unreachability with two checker rungs: the DRAT proof checked by `drat-trim` (independent, unverified), and — preferred when present — elaborated to LRAT (`drat-trim` as an *untrusted* elaborator) and re-validated by `cake_lpr`, the *formally verified* CakeML checker, built natively from its compiler-generated ARMv8 assembly (§6.5).

Interpreter adequacy (Assumption 1) is discharged differentially: the RISC-V interpreter against the Sail-generated reference simulator (`sail_riscv_sim`) and the official `riscv-tests` / `riscv-arch-test` suites; the Python subset against pinned CPython by construction — a circularity we note plainly: for this one language the pinned runtime is both the semantics and the interpreter, so adequacy has no independent reference; C against CBMC on the same source (with a classification step separating C-undefined-but-ISA-defined behavior from genuine faults).

5.3 Pairs and routes

Thirteen pairs are built to varying, *measured* extents (Table 2). The spine is C $\longrightarrow$ RISC-V $\longrightarrow$ BTOR2 $\longrightarrow$ SMT-LIB, with the C head a digest-pinned gcc at grade `reproducible`: the square-verified portion of the spine begins at the compiled binary (§4.2), and the C hop is re-checked separately by the CBMC differential above. The direct RISC-V→BTOR2 lowering draws on the rotor bounded model checker [8]. RISC-V and AArch64 each reach BTOR2 by two independently derived routes (manual-derived direct translator vs. Sail-model-mediated), instantiating Assumption 2; the BTOR2 hub funnels five ISAs into one solver interface, and the BTOR2 $\rightarrow$ SMT-LIB bridge (a rule-for-rule operator mapping at grade `predicted`) connects the hubs, so every BTOR2 question can also be decided natively-vs-bridged — solve-step corroboration. CRN and Python reach SMT-LIB directly (QF_LIA), and SMILES→formula exercises the calculus away from computation entirely.

5.4 How the pairs were built

Each pair (and each shared interpreter) was implemented by an *independent LLM agent* from a one-page registration brief, against the calculus as a written contract, under three standing rules: agents never modify the framework or another deliverable's code; interpreter changes are versioned events that re-validate every dependent pair (Proposition 4.7); and all growth is by additive widening — extend the covered fragment, re-run all prior evidence byte-for-byte, ratchet the coverage number. The platform's checks — the square oracle, the differentials, solver corroboration, branch agreement, twice-and-diff — were the only *semantic* quality gate; no human reviewed agent code for semantic correctness, though an autonomy protocol did escalate one class of change — edits to shared emission code — for human sign-off ("largely unsupervised" means exactly this: the human gated whether such a change could land, never whether it was right). The defects those checks caught, and where they localized, are part of the evaluation (§6.6): the implementation process itself is an experiment in the paper's thesis that translation trust can be manufactured by architecture rather than by trusting the translator's author — whether that author is a compiler vendor or a language model.

6 Evaluation

We evaluate the snapshot along the axes the calculus makes measurable: per-pair coverage (§6.1), composed route coverage and branch agreement (§6.2), a compliance-derived reachability benchmark decided along both RISC-V routes (§6.3), end-to-end case studies exercising Theorem 4.8 (§6.4), certified-unreachability runs inhabiting the `proved` tier (§6.5), the defects the architecture caught (§6.6), and performance and determinism (§6.9). Every number is regenerated by one script from the live registry and real runs at the snapshot commit (host: macOS/arm64, CPython 3.12, Z3 4.13, btormc 3.2, cadical; drat-trim pinned at source commit `2e3b2dc` and cake_lpr at `a4323b2`; the platform's own suite passes 1199 tests there). The evaluation remains

Table 2. Capability snapshot: per-pair construct coverage against the language-owned inventory. *Accepted* = the probe translates without a typed `unsupported` abort; *conjoined* = accepted *and* the pair's square oracle passes on the probe — Definition 4.6's conjunction, measured directly (§6.1). Pairs without a decidable square (the `predicted`-grade hops and the reproducible C hop) discharge the faithfulness conjunct per run instead (§4.2), marked *per-run*. Gaps counts the distinct typed `unsupported` constructs. Measured from the live registry at the snapshot commit.

Pair (source→target)	Grade	Accepted	Conjoined	Gaps
c-riscv	`reproducible`	—	—	—
riscv-btor2	`checked`	96/96	96/96	0
riscv-sail	`checked`	96/96	96/96	0
sail-btor2	`checked`	95/95	95/95	0
aarch64-btor2	`checked`	27/33	27/33	6
aarch64-sail	`checked`	27/33	27/33	6
wasm-btor2	`checked`	54/75	54/75	21
ebpf-btor2	`checked`	126/126	126/126	0
evm-btor2	`checked`	86/144	86/144	58
btor2-smtlib	`predicted`	56/56	per-run	0
crn-smtlib	`predicted`	10/10	per-run	0
python-smtlib	`predicted`	11/27	per-run	16
smiles-formula	`predicted`	14/17	14/17	3

deliberately *fully specified* rather than large: every question — authored or derived — is checked end to end with its ground truth stated, and the largest sweep (the 78-question compliance-derived benchmark of §6.3) is still small next to an SV-COMP-scale campaign, which remains future work.

6.1 Capability snapshot

Table 2 reports per-construct coverage measured at harvest time — not transcribed from documentation — in both of Definition 4.6's readings: *accepted* (the probe translates without a typed `unsupported` abort) and *conjoined* (accepted *and* the pair's square oracle passes on the probe — the definition's actual conjunction, run probe by probe). For every pair with a decidable square the two columns now coincide; they did not when the conjunction was first measured, and what it surfaced on its first run is instructive enough to report (§6.6, incidents I20–I22): the Sail-route front dropped the program's *initial memory*, so every load-family probe diverged at step 0 (accepted, silently wrong — precisely the failure mode acceptance counting cannot see), and all three BTOR2 lowerings modeled a program counter that leaves the code as *stuck* where every reference interpreter halts, so every taken-jump probe whose target exits the image diverged on `halted`. It also audited its own instrument: one probe (`SD`) stored over its own `ECALL` terminator, so the reference interpreter could not run it and no square had ever executed on that construct — acceptance counted it covered anyway. Measuring the definition's conjunction found all three in its first run; the fixes are ordinary versioned translator bumps and one probe repair. Pairs without a decidable square — the `predicted`-grade hops — discharge the faithfulness conjunct per run at question time instead (§4.2), which is exactly their grade's meaning; the table marks them *per-run*. Denominators are language-owned inventories (Definition 4.6's yardstick): both RISC-V-headed pairs measure against the same 96-construct RV64IMC inventory, the two AArch64 pairs against one declared A64 slice whose out-of-scope constructs count in the denominator, and the Wasm and EVM rows against their in-scope ∪ enumerated-out-of-scope unions (54/75, 86/144) — a gap shows as a typed `unsupported` entry, never by shrinking the total. Depth remains uneven by design and visibly so: the spine is deep (all 96 RV64IMC constructs conjoined; eBPF complete at 126; the BTOR2→SMT-LIB bridge total on its 56-item operator/-sort/directive inventory), Python stands at 11 of 27 subset constructs, and every gap is a *typed* `unsupported` the harness can enumerate (the "gaps" column). The declared A64 slice is still narrow against the full base ISA — the honest reading of 27/33 is "the in-scope family conjoins, and the slice is small." The C head carries no construct inventory: its translator is the pinned compiler, graded `reproducible` (§4.2).

6.2 Composed coverage and branch agreement

Table 3 (top) composes conjoined coverage along whole routes: a construct counts iff its probe survives *every* hop and every hop with a decidable square passes it on that hop's input. Nothing a head accepts is eaten at the hubs: every accepted probe of every route survives composed to SMT-LIB, squares included. The denominators are the source language's inventory, so the two RISC-V routes are measured on the *same* 96-construct yardstick and both compose 96/96 — the spine composes losslessly, and the earlier version of this table, whose two RISC-V rows quoted different pair-chosen denominators (96 vs. 95), is exactly the yardstick-shopping Definition 4.6 forbids and this measurement now closes. The AArch64 rows also carry a ratchet anecdote the machinery makes precise: one revision earlier the Sail-mediated route composed to 0/33, because the Sail→BTOR2 translator lowered only the RV64IMC arm — and the harness localized every miss to the `sail-btor2` hop by name. Widening that one hop (an additive translator bump, RISC-V arm byte-identical per Proposition 4.7) closed the gap without re-earning any other evidence. A gap the system *reports and localizes* is the designed behavior; a gap papered over would be the defect.

The bottom of Table 3 is the fidelity payoff of Lemma 4.5: the same reachability questions decided along independently derived routes. Twelve solver-level questions — constant dataflow, summation and countdown loops under unrolling, store/load through memory, two questions about a *gcc-compiled C program* (can `a0` be 47 at halt? can it be 99?), and four AArch64 questions decided along the Arm-manual-derived and the Arm-Sail-model-derived route — were each

Table 3. Composed *conjoined* coverage: a probe counts iff it survives every hop *and* every hop with a decidable square passes it on that hop's input (the route-level reading of Definition 4.6; "via Sail" routes are the independently derived branch; denominators are the source language's inventory, §6.2), and branch agreement: the same question decided along both routes. Times are the slower route, end to end (translate every hop + decide with Z3). The bottom block decides the same questions with fully disjoint stacks after the head: the direct route's BTOR2 system decided natively by btormc (no bridge, no SMT-LIB, no Z3) against the via-Sail route through the bridge and Z3 — the diverse segment spans both the lowering derivation and the decision procedure (§4.3).

Route (to SMT-LIB)	Composed coverage
RISC-V, direct	96/96
RISC-V, via Sail	96/96
AArch64, direct	27/33
AArch64, via Sail	27/33
Wasm	54/75
eBPF	126/126
EVM	86/144
CRN	10/10
Python	11/27
SMILES	14/17

Question (both routes)	Agree	Verdict	Time (s)
riscv const x1==42 (reach)	✓	reach	0.1
riscv const x1==99 (unreach)	✓	unreach	0.0
riscv loop sum==15 (reach)	✓	reach	0.0
riscv loop sum==99 (unreach)	✓	unreach	0.0
riscv store/load 0x123 (reach)	✓	reach	0.0
riscv store/load 0x999 (unreach)	✓	unreach	0.0
C a0==47 (reach)	✓	reach	0.1
C a0==99 (unreach)	✓	unreach	0.0
aarch64 movz/add x1==42 (reach)	✓	reach	0.0
aarch64 movz/add x1==999 (unreach)	✓	unreach	0.0
aarch64 SUBS/B.NE loop x0==1 (reach)	✓	reach	0.0
aarch64 SUBS/B.NE loop x0==5 (unreach)	✓	unreach	0.0
aarch64 flags + loop (SUBS/B.NE)	✓	trace $=_\pi$	—
aarch64 32-bit W forms	✓	trace $=_\pi$	—

Question, decision stacks fully disjoint	Agree	Verdict
riscv const x1==42 (reach)	✓	reach
riscv const x1==99 (unreach)	✓	unreach
riscv loop sum==15 (reach)	✓	reach
riscv loop sum==99 (unreach)	✓	unreach
riscv store/load 0x123 (reach)	✓	reach
riscv store/load 0x999 (unreach)	✓	unreach
aarch64 movz/add x1==42 (reach)	✓	reach
aarch64 movz/add x1==999 (unreach)	✓	unreach
aarch64 SUBS/B.NE loop x0==1 (reach)	✓	reach
aarch64 SUBS/B.NE loop x0==5 (unreach)	✓	unreach

Table 4. The compliance slice as an external-format question set: self-checking programs in the riscv-tests HTIF convention (built with the pinned toolchain at the upstream link base), each reference-run to derive per-register reachable / bounded-unreachable questions with machine-derived ground truth, each question decided independently along BOTH RISC-V→SMT-LIB routes. Agree = the two routes' verdicts coincide; correct = the agreed verdict matches the interpreter-derived ground truth. Time is both routes, all questions, end to end.

Program	Steps	k	Questions	Agree	Correct	Time (s)
`rv64uc-compress`	34	42	8	8/8	8/8	2.6
`rv64ui-arith`	51	59	8	8/8	8/8	5.5
`rv64ui-branch`	34	42	6	6/6	6/6	2.2
`rv64ui-jump`	18	26	8	8/8	8/8	1.3
`rv64ui-ldst`	68	76	8	8/8	8/8	11.3
`rv64ui-logic`	49	57	8	8/8	8/8	6.1
`rv64ui-shift`	45	53	8	8/8	8/8	5.1
`rv64ui-word`	43	51	8	8/8	8/8	5.0
`rv64um-div`	69	77	8	8/8	8/8	10.1
`rv64um-mul`	39	47	8	8/8	8/8	3.7
Total			78	78	78	

decided along both routes of their branch; all twelve pairs of verdicts agree, including the universal (UNREACHABLE) halves where agreement carries real information. The C rows exercise the full re-establishment story: an opaque `reproducible` compiler heads both routes, and its output is validated downstream twice, independently. The AArch64 trace-level rows cross-check the same branch below the solver, directly comparing the two carried-back behaviors under π. The table's bottom block addresses Assumption 2's scoping head-on: the twelve solver-level rows share the BTOR2→SMT-LIB bridge below the reconvergence, so ten of the questions are re-decided with fully disjoint stacks after the head — the direct route's BTOR2 system decided *natively* by btormc (no bridge, no SMT-LIB, no Z3) against the Sail-model route through the bridge and Z3. All ten agree. The diverse segment now spans the lowering derivation (ISA-prose vs. formal-model) *and* the decision procedure (different solver, different input format); the residual shared surface, stated rather than hidden, is the BTOR2 emission library — the site of incident I3, the platform's one recorded cross-route common-mode failure — and the language-owned endpoint interpreters.

6.3 The compliance slice as a question set

The questions above were authored; Table 4 scales the same discipline over a *derived* question set nobody hand-picked. The platform's RISC-V compliance slice — self-checking programs in the upstream riscv-tests HTIF `tohost` convention, built with the pinned cross-toolchain at the upstream link base `0x8000_0000` — doubles as a reachability benchmark:

Table 5. End-to-end case studies. "Replay" is the source-level witness check of Theorem 4.8: the carried-back witness re-executed by the source interpreter.

Case	Verdicts	Replay	Time (s)
C spine (both routes + source replay)	REACHABLE (both agree)	✓	0.06
Python assert violable? (QF_LIA)	REACHABLE	✓	2.99
EVM 6*7==42 native vs bridged	REACHABLE (both agree)	✓	0.05
CRN A->B twice reaches B=2	REACHABLE	✓	6.64

each program is reference-run to completion, and for each data register the harness derives one question whose target value the register demonstrably held (ground truth REACHABLE) and one whose target no register held at any step (ground truth UNREACHABLE within the bound), with the unrolling bound covering the whole run. Every question is then decided independently along *both* RISC-V routes. Two honesty notes. First, the slice is curated in the upstream convention rather than taken binary-for-binary from upstream: the stock `-p-` binaries open with machine-mode CSR and trap setup that is outside the declared user-ISA scope, so the slice re-creates the grading convention over the user subset (the gap is typed, not hidden). Second, the ground truth comes from the shared reference interpreter — but that interpreter is exactly the component the adequacy campaign validates step-for-step against the Sail-generated gold simulator *on these same programs*, so the benchmark's ground truth is anchored outside the platform. The result: 78 questions over 10 programs (constant dataflow, logic, shifts, 32-bit word ops, taken and untaken branches, computed jumps, the store/load family, multiplication, division with its RISC-V-defined edge cases, and compressed forms), both routes agreeing on all 78 and all 78 matching ground truth, in under a minute end to end. Preparing this benchmark also forced one more fidelity repair the base-0 probes could not see: the Sail-route front rebased images to address 0, which silently breaks every pc-relative absolute-address computation (`AUIPC`/`LA`) on images linked at the upstream base — found while squaring the translator against the slice's link discipline, and fixed by carrying absolute addresses through the Sail object (a versioned bump, with the squares and this benchmark as regression).

6.4 End-to-end case studies

Table 5 instantiates Theorem 4.8 once per hub and front-end family. (1) The *C spine*: both routes answer REACHABLE for `a0`=47, and the source-level replay runs the compiled program in the shared RISC-V interpreter to the halt in `_start()` with `a0`=47 observed — the verdict survives even if every translator and the solver were wrong. (2) *Python*: "is the trailing assert violable?" for a function with a bounded loop and a branch; Z3 finds the witness x=4, the shared QF_LIA evaluator independently re-checks the model (`smt_model_ok`), and the input is replayed through the pinned CPython down the taken branches, firing the assert. (3) *EVM*: 6×7=42 on the operand stack, decided *natively* (btormc on the BTOR2 system, witness replayed by the strict BTOR2 interpreter) and *bridged* (through the SMT-LIB hub with Z3) — solve-step corroboration across two engine families, agreeing. (4) *CRN*: a two-firing reachability question in a reaction network, witness schedule replayed by the Petri-net interpreter. In all four, the final trust rests on the source interpreter alone, per the last row of the ledger in §4.4.

Table 6. The certified (`proved`) row inhabited, twice: input-driven unreachability claims from a real pair, corroborated by three engines, certified by a bit-blasted DRAT proof, and re-validated by the independent verified checker. The first exhibit is deliberately small (its refutation is essentially unit propagation); the second is certificate checking at scale — refuting a 16×16-bit multiplication. The controls are bogus checks that must fail — and do.

	propagation-scale	**search-scale**
Question	$helper()^2 = 3$ (no square mod 2^{64})	$x \cdot y = 2^{31}-1$, $x, y \in [2, 2^{16}+1]$ (no bounded factorization of the Mersenne prime)
Engines agreeing UNSAT	bitwuzla, boolector, z3	bitwuzla, boolector, z3
Certificate (bitwuzla→CNF, cadical→DRAT)	41746 B DRAT, 18 B LRAT (drat-trim, untrusted)	2.8 MB DRAT, 11.8 MB LRAT
Independent check	cake_lpr (*formally verified*): **verified**; tier `proved`	cake_lpr (*formally verified*): **verified**; tier `proved`
Resulting TCB	bitwuzla:bit-blast, cake_lpr:verified	
Reachable sibling	x^2=4: REACHABLE, no certificate	$x \cdot y = 46341^2$: REACHABLE, no certificate
Negative controls	both rejected (bogus proofs vs. a satisfiable CNF)	
Wall time	0.22 s	4.64 s

6.5 The certified tier

Table 6 inhabits the ledger's certified row (§4.4), twice — once at each end of the difficulty scale. Both questions are pair-derived and input-driven: eBPF programs whose `CALL` models helper returns as free inputs. The first squares one input and asks whether $x^2 = 3$; no square exists modulo 2^{64}, so the claim is universal over all inputs — and its refutation is essentially unit propagation on the low bits (the elaborated LRAT is 18 bytes). The second masks and offsets two inputs into $[2, 2^{16}+1]$, multiplies them, and asks whether the product can be $2^{31} - 1$: the Mersenne prime admits no factorization with both factors in range, and refuting a 16×16-bit multiplication is certificate checking at scale — a 2.8 MB

DRAT elaborating to an 11.8 MB LRAT, re-validated in seconds. In both, three engines corroborate UNSAT, bitwuzla bit-blasts the query to CNF, cadical emits a DRAT refutation, `drat-trim` elaborates it to LRAT — an *untrusted* step: a wrong elaboration can only fail the re-check, never fake a pass — and `cake_lpr`, the *formally verified* CakeML LRAT checker (soundness machine-proved down to the binary; here built natively from its compiler-generated ARMv8 assembly), re-validates the proof against the CNF from scratch. The verdicts' TCB is exactly the ledger's certified row: the bit-blaster's BV→CNF step and a verified checker, with all three deciding engines and the elaborator demoted to discovery devices. The reachable siblings ($x^2 = 4$; $x{\cdot}y = 46341^2$) correctly decline to certify.

Wiring this exhibit produced one more architecture-caught defect, live: the checker *adapter* matched the substring `VERIFIED` in the checker's output — and `drat-trim` reports failure as `s NOT VERIFIED`, which contains it. Every check, sound or bogus, reported success. The bug could not surface on hosts without the checker (the call raised before parsing) nor on real certificates (which do verify); it fell to the *negative control* — a bogus refutation of a satisfiable CNF that must fail and did not. The fix parses the exact status line, and the control is now a permanent test. This is the paper's own instrument audited by the discipline it describes (cf. §6.6), and a caution we commend to anyone wiring a proof checker: *a checker adapter without a negative control is itself unchecked.* The verified checker's adapter was written under that rule from the start — wisely, since `cake_lpr` exits 0 even when checking fails; only the exact `s VERIFIED UNSAT` line signals success, and both checkers' negative controls are permanent tests. One honest subtlety the exercise surfaced: for this instance the refutation is propagation-dominated, so mutating single proof lines is not a valid negative control (the checker legitimately completes the derivation); the satisfiable-formula control is the sound one, since no valid refutation of a satisfiable formula exists.

6.6 Defects the architecture caught

The pairs were written by independent LLM agents with no human semantic review (§5.4); the architecture's checks were the only gate. Mining the full history (675 commits across all refs; ~21 runtime-code fix commits) plus the three defects the conjoined-coverage measurement caught on its first run (§6.1) yields 22 recorded defect incidents: 15 caught by the architecture's cross-checks rather than by ordinary unit tests, and 5 more — including the checker-adapter defect of §6.5 and a probe that could never have exercised its own square — by the audits and controls the architecture imposes on its own instruments. Read this as case-study evidence, not a hit rate: the catalog is mined from our own history, it has no denominator (the blind-spot incident below proves the escape rate is nonzero), and no baseline says what a conventional test regime would have caught. Table 7 lists ten

Table 7. Defects caught by the architecture (selection; full list with commit-level evidence in the artifact). "Caught by" names the layer of §4.4 that fired.

Defect	Component	Caught by
bv64 sort hardcoded for `and`/`or`/`xor` over bv1 operands — malformed BTOR2 that real solvers tolerated	RISC-V → BTOR2 translator	strict in-process BTOR2 evaluator during the square's alignment walk
unsigned loads (`LBU`/`LHU`/`LWU`) missing zero-extension to bv64	RISC-V → BTOR2 translator	solver leg: Z3 sort rejection on the corpus
`init` value nid emitted after the state nid — rejected by every conformant tool, tolerated by the one wired solver; affected every stateful pair	shared BTOR2 emitter	solver corroboration: wiring the 2nd/3rd engines (btormc, pono) made the latent malformation manifest
`local.get` wrote a bv32 node into the bv64 stack array	Wasm → BTOR2 translator	solver leg: Z3 sort mismatch at BMC
carry-back silently zero-filled registers the solver witness omits (init-pinned states) — every pinned task's entry state misread	witness carry-back Λ	witness-replay anchor audit (mismatch chain)
default BMC bound mapped "no violation within k" to UNREACHABLE — a false negative at step 93	verdict semantics	oracle vs. corpus label
ELF header bytes decoded as instructions — interpreter tolerated it, the square did not	shared ELF loader	square oracle on the first real gcc binary
BTOR2 `constraint` lines silently dropped in the SMT-LIB unrolling — an under-constrained, soundness-leaking encoding	BTOR2 → SMT-LIB bridge	coverage probe: driving the bridge to its 56/56 spec inventory exposed the hole
`JUMP`/`JUMPI` underflow-halt row diverged on pc	EVM → BTOR2 translator	square step alignment during widening
`INT_MIN / -1`: C-undefined vs. RV64-defined divergence, plus a CBMC false positive on the same task	(real route divergence)	C-vs-ISA differential branch, with a UB-vs-fault classifier adjudicating

representative architecture-caught defects, each localized — component, step, observable — as Corollary 3.8 promises.

Three second-order observations are worth as much as the table. First, *the checks caught their own instruments*, three times: the Sail ISA differential was once passing vacuously (comparing empty traces), the in-image suite caught a witness generator emitting empty witnesses, and the DRAT checker adapter accepted every outcome until a negative control ran (§6.5) — all found because the harnesses carry positive *and negative* controls, and all three are why Assumption 1 is stated as an assumption with empirical discharge rather than as a fact. Second, the history contains a genuine *blind spot* of the square, exactly of the class Lemma 4.5

Table 8. Escape-rate experiment: seeded semantic mutations of the riscv-btor2 emissions (uniform rules model systematic mis-lowerings, site rules single-site defects; operand swaps only on non-commutative operators; rules that change no probe artifact are excluded from the denominator), each run through the architecture's gates in order. "Square" is the conjoined probe suite (§6.1), "branch" the authored solver questions against the intact Sail route, "bench" the compliance-derived ground-truth questions (§6.3).

Applicable mutants	55
Caught by the square suite	51
Caught by branch agreement	0
Caught by the derived benchmark	4
Escaped all three gates	**0**

predicts: the RISC-V interpreter *and* the RISC-V→BTOR2 translator both silently mis-decoded `MUL` (funct7 $0x01$) as `ADD` — both legs agreed, the square was blind, and the defect was found by manual audit. The platform's structural answer is diversity: the decoder is now anchored against 610 Sail-emitted words, and unknown funct7 hard-aborts on both sides. Third, in one incident three existing unit tests had been written *against the buggy behavior* (an evaluator masking every array store to 8 bits): tests written by a component's author codify the author's misreading; cross-checks against independently derived semantics do not. The architecture also *disconfirmed* one reported bug (a claimed slice-width defect shown to be a stale-cache artifact), and its differential campaigns — 300 seeded RV64IMC programs vs. the Sail simulator, a 10-seed Csmith campaign vs. native gcc, 463 reference cases — completed with zero divergences against harnesses whose positive controls confirm they can fail.

6.7 An escape-rate estimate for the gate

The catalog above is mined from history and has no denominator; the MUL/ADD blind spot proves the gate's escape rate is nonzero. Table 8 measures it: seeded semantic mutations of the `riscv-btor2` translator's emissions — uniform rules modeling systematic mis-lowerings (every `sext` emitted as `uext`, incident I2's family) and site rules modeling single-site defects (incident I1's family); operand swaps only on non-commutative operators, so no trivially-equivalent mutants; rules that change no probe artifact excluded from the denominator — each run through the architecture's gates in the order the platform applies them. The experiment earned its keep before its final numbers did: in its first round, both `srl`→`sra` mutants escaped *all three* gates (36 of 51 caught at the square layer), because every ALU probe ran its construct on degenerate all-zero operands, on which logical and arithmetic shifts agree — the "96/96 conjoined" verdict was faithfulness *on the probes*, and the probes underdetermined the constructs. Hardening the probes with mixed-sign operands moved 50 of 53 catches to the square layer but let `ult`→`ulte` escape — strictness is observable only at *equal* operands — so the probes now run an equal-operand and a mixed-sign instance per construct (incident I23; the ratchet kept every previously covered construct covered throughout). In the final round, 51 of 55 applicable mutants die at the square suite, the remaining 4 at the compliance-derived benchmark, and none escape. Three honest notes: zero escapes means this *mutation family* is now covered, not that the escape rate is zero — single-leg mutation cannot model the MUL/ADD class where both legs of the square are wrong identically; the authored branch questions caught nothing in any round (the derived benchmark subsumes them as a differentiator, a finding in itself); and the denominator is one translator's emission space, not the platform.

6.8 The other direction: an LLM plays the platform

The introduction's two-directional experiment has, until here, reported only its build half. This subsection is a first, deliberately small evaluation of the *player* half: 12 reachability questions with platform-established ground truth (6 reachable / 6 unreachable, spanning RISC-V signedness and load-width traps, eBPF square-residue and bounded-factorization questions, and Python assert-violability), posed to fresh, context-free frontier-LLM agents under two protocols. Arm A (unaided) received only the question text, instructed to answer by reasoning with no tools. Arm B (player) received the same question plus the repository and the head-construction snippet, instructed to decide *via the platform* and report the verdict with its evidence basis. One agent per question per arm, no retries; prompts, ground truths, per-run outputs, and grading are in the artifact (`results/llm_player/`).

Both arms scored 12/12. That is the honest headline, and it reframes what the platform buys at this question scale: not raw accuracy — the unaided frontier model simulated the loops, caught both signedness traps, and factored 2147766287 by Fermat's method in its head — but the *evidence class* of the answer. Every arm-A verdict rests on the model's say-so (all were reported at high confidence, so confidence would not have flagged a wrong one); every arm-B verdict carries a machine-checked artifact: two-route agreement on the RISC-V questions, hedged explicitly as bounded-k verdicts resting on route fidelity; witness replays through the strict interpreters on the reachable eBPF/Python questions; and tier-raises to `proved` on the unreachable eBPF questions. The sharpest contrast is the bounded-factorization question: arm A answered correctly by *recalling* that 1073741789 is the largest prime below 2^{30} — opaque memory, exactly the evidence the calculus cannot grade — while arm B produced a 17 MB LRAT certificate re-validated by the formally verified checker, with the trusted base named. One player even devised its own positive control unprompted (flipping the load question's target to the sign-extended value and confirming REACHABLE on both routes). Three limitations, stated plainly:

Table 9. Route cost for the RISC-V loop question (k=25 unrolling): whole-route translation, cold vs. warm (the content-addressed cache of Proposition 3.9), Z3 decide time, byte-determinism (twice-and-diff), and artifact size.

Route	cold (ms)	warm (ms)	decide (ms)	det.	artifact
RISC-V, direct	5	0.0	32	✓	201 kB
RISC-V, via Sail	5	0.0	33	✓	209 kB

Table 10. Scalability probes on two axes. Left: the summation-loop question at growing bounds ($x_1 = N(N+1)/2$ after N iterations; reachable at every size), whole-route translation and Z3 decide time. Right: the certified tier on the bounded-factorization family at growing factor widths — certificate sizes grow four orders of magnitude and cake_lpr re-validates each.

Loop question	verdict	translate (s)	decide (s)	artifact
N=5, k=30	reachable	0.01	0.04	240 kB
N=20, k=105	reachable	0.02	0.10	830 kB
N=50, k=255	reachable	0.04	0.27	2054 kB
N=100, k=505	reachable	0.07	0.55	4093 kB

Certified question	outcome	DRAT	LRAT	time (s)
8-bit factors, target 65521	certified	27.3 kB	24.7 kB	0.2
12-bit factors, target 16777213	certified	0.2 MB	64.1 kB	0.3
16-bit factors, target 2147483647	certified	2.8 MB	11.8 MB	4.9

the protocol scripted which platform entry points to use (the players executed and interpreted; they did not discover the platform cold); the subject model is from the same family that built the platform; and 12 questions at this size cannot separate the arms on correctness — questions hard enough that unaided reasoning actually fails are the obvious next step.

6.9 Performance, determinism, and scale

Translation is not the cost center: whole-route translation of the loop question (k=25 unrolling, ∼190 kB of SMT-LIB) is ∼5 ms cold and a sub-millisecond content-addressed cache hit warm (Proposition 3.9 cashed in), against a Z3 decide of 32 ms on either route; the twice-and-diff determinism check passes on both routes. The numbers say the honest overhead of the whole discipline — checking every hop, re-running for determinism, replaying witnesses — is small against the solver call it wraps. Table 10 probes where the pipeline stands as questions grow, on two axes: unrolling the summation loop to N=100 (k=505, a 4 MB artifact) keeps whole-route translation under 0.1 s and the decide under 1 s, growing roughly linearly; and the certified tier survives the bounded-factorization family from propagation-size to search-size — certificates growing from 25 kB to an 11.8 MB LRAT, each re-validated by the verified checker in seconds. Neither axis reaches industrial scale, but both cross the region where "toy" criticisms live: the pipeline's costs grow smoothly, with no cliff through the measured range.

7 Related work

Certified compilation and translation validation. CompCert [13] and CakeML [12] prove one translator correct once; translation validation [21, 25] and its modern descendants [14, 33] validate each run of one translator, and validators can themselves be verified [37]. Credible compilation [27] — per-run validation of a compiler's own result — is a direct ancestor of re-establishment (Theorem 4.3). In our vocabulary these are grades of a *single edge*; our contribution is the graph of edges of heterogeneous grade — how faithfulness, loss, and assurance compose along routes, how per-run validation re-establishes hops that carry squares, and how independent routes corroborate.

Compositional compiler correctness. Composing simulations across passes and languages is a rich line: Compositional CompCert [34], parametric inter-language simulations [22], CompCertO's simulation conventions [11], certified abstraction layers [5], multi-language semantics [15, 24], and DimSum's decentralized multi-language graph with wrappers [31]. These works confront vertical composition of simulations *up to differing observations* — the habitat of our support condition, which in their terms is the congruence/well-definedness requirement on the mediating map; CompCert sidesteps it by fixing one global event type, CompCertO negotiates it through simulation conventions, DimSum through wrappers. We claim no mathematical novelty for the condition (Definition 3.6 is a one-line congruence); what this paper adds relative to that line is a different regime and different bookkeeping: edges that are *not* uniformly proved, trace-equality squares that are *decidable per run* rather than proved once, an explicit syntactic keep/loss computation for routes, and the grade/class algebra for composing heterogeneous evidence — with the trade-off stated plainly, since equality-up-to-projection is the right notion only for near-structure-preserving translations, not optimizing compilation (§3.1).

Semantics-first frameworks. The K framework derives interpreters, symbolic executors, and provers from one semantics [28], with KEVM [6] and KWasm as flagship instances; Sail does the same for ISAs [2]. These are our closest philosophical relatives, and our platform *consumes* their artifacts (the Sail RISC-V and Arm models anchor one of each ISA's two routes). The difference is what carries trust: a semantics-first framework is a monoculture — every derived tool inherits the one semantics and the one derivation pipeline — whereas our unit of trust is *agreement between independently derived translations* (Assumption 2), with the framework structured so routes diverge over independent semantic artifacts. N-version programming [3] and differential testing [17, 39] exploit the same independence resource — and Knight and Leveson's experiment [10] is the standing caution that independence must be argued, not assumed, which is why Assumption 2 is explicit, scoped to the diverse prefix, and confronted with our own common-mode incidents (§6.6). We give the possibilistic account of what agreement does and does not establish (Lemma 4.5) and wire it into a compositional calculus rather than a test harness.

Certificates, witnesses, and certifying algorithms. Proof-carrying code [20], certifying algorithms [16], certified model-checking [19, 23, 40], and checked solver proofs (DRAT [38], verified checkers [36], Alethe/Carcara [1]) all instantiate the producer/checker split our `proved` grade requires, and SV-COMP's violation/correctness-witness ecosystem [4] is the deployed, cross-tool form of the existential/universal asymmetry our end-to-end theorems state. What we add is the composition through a translation graph: an existential answer's certificate is its *source-level replay* after carry-back (Theorem 4.8), while universal answers compose certificate checking with route fidelity (Theorem 4.9). The carry-back itself has verified precedent — decompilation into logic [18] makes the lifter a proved component — and our contribution is making it a mandatory, per-pair component of every edge rather than a per-tool artifact.

Moving programs to answerable representations. Verified lifting [7] and the broad practice of encoding programs into solver logics (BMC, symbolic execution) motivate the platform's existence; refinement stacks such as seL4's [9, 33] show multi-layer semantic preservation with uniform, proof-backed layers. Our setting differs in admitting layers that are *not* uniformly proved — pinned compilers, agent-written translators, third-party models — and making their heterogeneous trust first-class rather than a threat to the statement. Combining heterogeneous verification evidence with recorded provenance is itself prefigured by the Evidential Tool Bus [29]; the grade/class algebra and the ledger are a formalized cousin.

Fault tolerance, end to end. The paper's vocabulary is deliberately that of fault-tolerant systems, because the concepts coincide: the square oracle is an acceptance test in the recovery-block sense [26], making a silently wrong — in effect Byzantine — translator fail-stop [32]; a branch is dual modular redundancy whose comparator localizes rather than votes; and the existential/universal asymmetry is the end-to-end argument [30] for translation graphs. What the transplant adds is the compositional bookkeeping — graded evidence with weakest-link meets, per-run re-establishment, and the trusted-base ledger.

LLMs and formal tools. Recent systems couple LLM generation with solver-checked validation [35]. Our platform is built for that player — an LLM chooses questions and routes but every step it takes is deterministic, graded, and checked — and §5.4 reports the build half of the two-directional experiment of Section 1: the translators themselves were LLM-written, and correctness was carried by the architecture, not by author trust. §6.8 reports a first controlled experiment on the player half.

8 Limitations and conclusion

Limitations. This is a dated snapshot of a system designed to ratchet, and its numbers say so: coverage is deepest on the spine (all 96 RV64IMC constructs conjoined and composing losslessly to SMT-LIB on both routes; the in-scope eBPF set complete) and deliberately partial elsewhere (EVM at 86 of 144 opcodes; Wasm at 54 of its 75-item declared inventory, without `loop`/`br`; Python an integer subset with bounded unrolling). Coverage is now measured as Definition 4.6's actual conjunction over language-owned inventories wherever a pair has a decidable square (§6.1); the `predicted`-grade hops still discharge faithfulness per run rather than per construct, and the A64 inventory, while language-owned, is a declared slice of a much larger ISA. Universal answers are bounded claims wherever BMC-style unrolling caps apply, and rest on universal-tier hypotheses that per-run evidence corroborates but does not entail (Theorem 4.9). The verified portion of the C spine begins at the compiled binary: the compiler hop has no square and contributes reproducibility plus a differential, not faithfulness (§4.2). The `proved` tier's certified row is inhabited with a *formally verified* checker at the anchor (`cake_lpr`, §6.5), on one trivial and one certificate-scale instance (an 11.8 MB LRAT); the residual TCB is the bit-blaster's BV→CNF step, and a certified claim is per-question and bounded, never a proof of a pair. Interpreter adequacy remains an assumption discharged empirically at modest volume, and the diversity assumption behind branch corroboration is structural, not probabilistic — and scoped to the diverse prefix. The compositional core of the calculus — including the n-ary route telescope and the specialization obligation of Theorem 4.9 — is mechanized in Lean 4 with a clean axiom audit (§4.7); the fieldwise loss computation and the retiming case remain paper-stated.

Future work. The gaps above are the roadmap: scale the compliance-derived benchmark of §6.3 to an upstream suite taken binary-for-binary (which needs the machine-mode slice) and to SV-COMP-style C tasks; extend the escape-rate experiment of §6.7 beyond one translator's emission space and to mutation families that model common-mode (both-leg) defects; widen the A64 slice toward the base ISA; and scale the player experiment of §6.8 to questions hard enough that unaided reasoning measurably fails, with subject models outside the builders' family.

Conclusion. We proposed treating translations the way distributed systems treat processes: as components that fail, whose failures are contained by architecture — declared projections, decidable squares, graded evidence, independent routes, carried-back witnesses — rather than prevented by universal proof. The calculus makes the composition of partial trust exact; the platform shows the discipline is buildable, by agents, at measured (if modest) coverage; and the asymmetry theorems say where proof effort actually buys assurance. The snapshot is the baseline; the ratchet is the roadmap.

References

[1] Bruno Andreotti, Hanna Lachnitt, and Haniel Barbosa. 2023. Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. In *TACAS*.
[2] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 71:1–71:31.
[3] Algirdas Avižienis. 1985. The N-Version Approach to Fault-Tolerant Software. *IEEE Transactions on Software Engineering* SE-11, 12 (1985), 1491–1501.
[4] Dirk Beyer, Matthias Dangl, Daniel Dietsch, Matthias Heizmann, and Andreas Stahlbauer. 2015. Witness Validation and Stepwise Testification across Software Verifiers. In *ESEC/FSE*.
[5] Ronghui Gu, Jérémie Koenig, Tahina Ramananandro, Zhong Shao, Xiongnan (Newman) Wu, Shu-Chun Weng, Haozhong Zhang, and Yu Guo. 2015. Deep Specifications and Certified Abstraction Layers. In *POPL*.
[6] Everett Hildenbrandt, Manasvi Saxena, Nishant Rodrigues, Xiaoran Zhu, Philip Daian, Dwight Guth, Brandon Moore, Daejun Park, Yi Zhang, Andrei Stefanescu, and Grigore Roşu. 2018. KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine. In *CSF*.
[7] Shoaib Kamil, Alvin Cheung, Shachar Itzhaky, and Armando Solar-Lezama. 2016. Verified Lifting of Stencil Computations. In *PLDI*.
[8] Christoph M. Kirsch. 2017. Selfie and the Basics. In *Onward!*
[9] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *SOSP*.
[10] John C. Knight and Nancy G. Leveson. 1986. An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 96–109.
[11] Jérémie Koenig and Zhong Shao. 2021. CompCertO: Compiling Certified Open C Components. In *PLDI*.
[12] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *POPL*.
[13] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115.
[14] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*.
[15] Jacob Matthews and Robert Bruce Findler. 2007. Operational Semantics for Multi-language Programs. In *POPL*.
[16] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161.
[17] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
[18] Magnus O. Myreen, Michael J. C. Gordon, and Konrad Slind. 2008. Machine-Code Verification for Multiple Architectures: An Application of Decompilation into Logic. In *FMCAD*.
[19] Kedar S. Namjoshi. 2001. Certifying Model Checkers. In *CAV*.
[20] George C. Necula. 1997. Proof-Carrying Code. In *POPL*.
[21] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *PLDI*.
[22] Georg Neis, Chung-Kil Hur, Jan-Oliver Kaiser, Craig McLaughlin, Derek Dreyer, and Viktor Vafeiadis. 2015. Pilsner: A Compositionally Verified Compiler for a Higher-Order Imperative Language. In *ICFP*.
[23] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *CAV*.
[24] James T. Perconti and Amal Ahmed. 2014. Verifying an Open Compiler Using Multi-language Semantics. In *ESOP*.
[25] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *TACAS*.
[26] Brian Randell. 1975. System Structure for Software Fault Tolerance. *IEEE Transactions on Software Engineering* SE-1, 2 (1975), 220–232.
[27] Martin Rinard and Darko Marinov. 1999. *Credible Compilation*. Technical Report MIT-LCS-TR-776. MIT Laboratory for Computer Science.
[28] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434.
[29] John Rushby. 2005. An Evidential Tool Bus. In *ICFEM*.
[30] Jerome H. Saltzer, David P. Reed, and David D. Clark. 1984. End-to-End Arguments in System Design. *ACM Transactions on Computer Systems* 2, 4 (1984), 277–288.
[31] Michael Sammler, Simon Spies, Youngju Song, Emanuele D'Osualdo, Robbert Krebbers, Deepak Garg, and Derek Dreyer. 2023. DimSum: A Decentralized Approach to Multi-language Semantics and Verification. *Proceedings of the ACM on Programming Languages* 7, POPL (2023).
[32] Richard D. Schlichting and Fred B. Schneider. 1983. Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems. *ACM Transactions on Computer Systems* 1, 3 (1983), 222–238.
[33] Thomas Arthur Leck Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. In *PLDI*.
[34] Gordon Stewart, Lennart Beringer, Santiago Cuellar, and Andrew W. Appel. 2015. Compositional CompCert. In *POPL*.
[35] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2024. Clover: Closed-Loop Verifiable Code Generation. *arXiv preprint arXiv:2310.17807* (2024).
[36] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. 2021. cake_lpr: Verified Propagation Redundancy Checking in CakeML. In *TACAS*.
[37] Jean-Baptiste Tristan and Xavier Leroy. 2008. Formal Verification of Translation Validators: A Case Study on Instruction Scheduling Optimizations. In *POPL*.
[38] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *SAT*.
[39] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*.
[40] Emily Yu, Armin Biere, and Keijo Heljanko. 2021. Progress in Certifying Hardware Model Checking Results. In *CAV*.

A Proofs

A.1 Pasting (Theorem 3.7)

We spell out the chain, including the domain side conditions elided in the body. Let $p \in \mathrm{dom}(P_2 \circ P_1)$, i.e. $p \in \mathrm{dom}(I_A) \cap \mathrm{dom}(T_1)$, $q := T_1(p) \in \mathrm{dom}(T_2)$, $r := T_2(q)$, $I_C(r)$ defined, $\Lambda_2(I_C(r)) \in \mathrm{dom}(\Lambda_1)$, and (from $p \in \mathrm{dom}(P_1)$,

$q \in \mathrm{dom}(P_2)$) $I_B(q)$ defined with $I_B(q) \in \mathrm{dom}(\Lambda_1)$ and $I_C(r) \in \mathrm{dom}(\Lambda_2)$.

1. By faithfulness of P_1 at p and $\pi \subseteq \pi_1$, $\pi(I_A(p)) = \pi(\Lambda_1(I_B(q)))$.
2. By faithfulness of P_2 at q: $\pi_2(I_B(q)) = \pi_2(\Lambda_2(I_C(r)))$.
3. Both $I_B(q)$ and $\Lambda_2(I_C(r))$ lie in $\mathrm{dom}(\Lambda_1)$ (the first by $p \in \mathrm{dom}(P_1)$, the second by $p \in \mathrm{dom}(P_2 \circ P_1)$). By $(\pi_2 \Rightarrow \pi)$-support applied to step (2): $\pi(\Lambda_1(I_B(q))) = \pi(\Lambda_1(\Lambda_2(I_C(r))))$.
4. Chaining (1) and (3): $\pi(I_A(p)) = \pi(\Lambda_1(\Lambda_2(I_C(r))))$, which is faithfulness of $P_2 \circ P_1$ at p w.r.t. π. □

Necessity of support. Without Definition 3.6 the theorem is false: let $F_B = \{g, h\}$, $\pi_2 = \{g\}$, and let Λ_1 copy field h into a π_1-kept source field. Take P_2 faithful (it preserves g) but with $\Lambda_2(I_C(r))$ differing from $I_B(q)$ on h — permitted, since $h \notin \pi_2$. The outer rectangle then disagrees on the copied field although both inner squares commute.

A.2 Composed keep-set, and step granularity

For fieldwise, step-aligned Λ_1 (each kept source field f computed per step as $f = \phi_f(\{\, b.g \mid g \in \mathit{deps}(f)\,\})$), $(\pi_2 \Rightarrow \pi)$-support holds iff $\mathit{deps}(f) \subseteq \pi_2$ for every $f \in \pi$; hence the largest admissible π is keep $= \{f \in \pi_1 \mid \mathit{deps}(f) \subseteq \pi_2\}$ and support for it is checkable syntactically from the dependency sets.

When Λ_1 retimes — one source step corresponds to a window of target steps, as in C statements over ISA instructions — model Λ_1 as a monotone window map w from source step indices to intervals of target step indices plus a fieldwise read within each window. Support then requires $\mathit{deps}(f) \subseteq \pi_2$ for reads *anywhere in the window*, and additionally that the window map itself be determined by π_2-observables (e.g. by a kept program counter), since window boundaries influence the projected output. Both conditions are again syntactic given w and the dependency sets.

A.3 Localization (Corollary 3.8)

Contrapositive of Theorem 3.7 for two hops. For n hops, induct on the route: if the outer rectangle over hops $1..n$ fails at p but hop 1's square passes at p, then by the two-hop case applied to (hop 1, rectangle over hops $2..n$) the rectangle over $2..n$ fails at $T_1(p)$; recurse. Termination yields a least failing hop i; within it, the square oracle's comparison yields the least failing step and a witnessing field. □

A.4 Weakest link (Proposition 4.2)

Soundness: if every hop is universal on its fragment, iterate Theorem 3.7 over all p in the composite fragment. If some hop is perrun, the same iteration is available exactly at programs whose per-hop images passed the oracle — the per-run set of the route. If some hop is only replay, purity composes (Proposition 3.9) but no faithfulness statement is available for the route beyond it. Optimality: a route with a perrun hop entails no universal statement (take a defect outside the checked set); a route with a replay hop entails no faithfulness at all (take a pure but wrong translator). □

A.5 Ratchet (Proposition 4.7)

Let $I' \sqsupseteq I$ and let $p \in \mathrm{dom}(I)$. Every quantity in Definition 3.5 evaluated at p reads only I-values on $\mathrm{dom}(I)$-points, which I' preserves; hence the verdict at p is unchanged. Coverage monotonicity: cov's per-construct conjunction can only gain (new constructs enter dom) and never lose (old verdicts preserved). Composition of extensions is an extension. For the failure half: a non-extension update changes some $I(p)$, which can flip the verdict at p for any pair whose square consumed it — justifying the versioned-event rule. □

B Construct inventories and probe corpora

Per-language construct inventories (the yardsticks of Definition 4.6) and the probe corpora behind every coverage figure in Section 6 are enumerated in the artifact (`results/`); they are spec-derived: the RISC-V RV64IMC opcode inventory, the full 144-opcode EVM inventory, the 256-opcode eBPF inventory slice, the Wasm numeric/parametric/control inventory, the OpenSMILES construct list, the Python-subset AST allow-list, and the two hubs' operator inventories.